

LAPORAN PRAKTIKUM

TUGAS 3 Docker Network, Volume, Bind Mount, tmpfs, dan Compose

Dosen Pengampu : Ferry Astika Saputra



Disusun Oleh :

Adam Rasyid Nurmuhammad

3126640025

D4-IT-B

**PROGRAM STUDI SARJANA TERAPAN JURUSAN TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER POLITEKNIK
ELEKTRONIKA NEGERI SURABAYA 2026**

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan	3
BAB II LANDASAN TEORI	4
BAB III Pengerjaan Praktikum.....	5
3.1 Persiapan Direktori Kerja dan Status Docker	5
3.2 User-Defined Bridge Network dan DNS Service Discovery	6
3.3 Manajemen Data Container: Named Volume, Bind Mount, dan tmpfs	6
3.4 Implementasi Stack Multi-Container dengan Docker Compose	7
3.5 Verifikasi dan Pengujian Layanan	8
3.6 Analisis Wajib dan Troubleshooting	8
BAB IV EVALUASI DAN LATIHAN MANDIRI	10
BAB V KESIMPULAN	11

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam lanskap rekayasa perangkat lunak modern dan metodologi DevSecOps, penerapan arsitektur modular seperti multi-container mikroservis telah menjadi standar de-facto. Sistem production umumnya mengisolasi fungsi web server/reverse proxy, aplikasi backend, dan basis data ke dalam container independen. Konfigurasi ini menuntut pemahaman mendalam mengenai segregasi jaringan, penjaminan persistensi data, serta orkestrasi deklaratif siklus hidup kontainer. Docker menyediakan instrumen krusial melalui user-defined bridge network untuk komunikasi antarlayanan yang terisolasi dengan DNS otomatis, serta opsi storage mount (Named Volume, Bind Mount, tmpfs) yang disesuaikan dengan kebutuhan persistensi dan keamanan data. Selain itu, Docker Compose menyatukan orkestrasi multi-container secara reproduktif dan terkelola.

1.2 Tujuan

1. Mampu membuat user-defined bridge network dan membuktikan name resolution otomatis antar container.
2. Membedakan karakteristik, persistensi, portabilitas, dan keamanan antara volume, bind mount, dan tmpfs.
3. Mampu melakukan proses pencadangan (backup) volume data container secara terisolasi menggunakan helper container.
4. Menyusun deklarasi Docker Compose multi-tier (Nginx-Flask-PostgreSQL) dengan pembagian network frontend dan backend.
5. Menerapkan mekanisme service healthcheck dan dependency condition untuk menjamin kesiapan runtime.
6. Mendiagnosis kendala build aplikasi serta merumuskan analisis risiko keamanan dan rekomendasi production

BAB II LANDASAN TEORI

Docker network berfungsi sebagai boundary arsitektural yang membentuk graf keterjangkauan (reachability graph) antar komponen. Driver bridge menghubungkan container pada satu host menggunakan virtual network interfaces. Berbeda dengan default bridge bawaan Docker, user-defined bridge network menyediakan isolasi yang ketat dan fitur automatic DNS service discovery, di mana container dapat saling berkomunikasi melalui nama kontainer atau nama service alih-alih IP dinamis yang rentan berubah. Port publishing (-p) hanya boleh diterapkan pada service ingress publik, sedangkan komponen sensitif seperti database tetap internal di backend network guna memenuhi prinsip least exposure

BAB III Pengerjaan Praktikum

3.1 Persiapan Direktori Kerja dan Status Docker

1. Pembuatan Direktori Kerja Bab 3

Membuat direktori kerja terisolasi ~/docker-lab/bab-3 menggunakan perintah mkdir -p dan masuk ke direktori tersebut

```
adam@Devsecops:~$ mkdir -p ~/docker-lab/bab-3
adam@Devsecops:~$ cd ~/docker-lab/bab-3
adam@Devsecops:~/docker-lab/bab-3$ pwd
/home/adam/docker-lab/bab-3
adam@Devsecops:~/docker-lab/bab-3$ ls -la
total 8
drwxrwxr-x 2 adam adam 4096 Sep 12 09:12 .
drwxrwxr-x 5 adam adam 4096 Sep 12 09:12 ..
```

2. Verifikasi Kesiapan Docker Engine

Memeriksa status daemon dan integritas engine melalui docker version dan docker ps sebelum memulai praktikum

```
adam@Devsecops:~/docker-lab/bab-3$ docker version
Client: Docker Engine - Community
 Version:           29.8.0
 API version:       1.56
 Go version:        go1.26.8
 Git commit:        88096ef
 Built:             Thu Sep  3 21:50:45 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:           29.8.0
  API version:       1.56 (minimum version 1.40)
  Go version:        go1.26.8
  Git commit:        3ce5872
  Built:             Thu Sep  3 21:50:45 2026
  OS/Arch:           linux/amd64
  Experimental:      false
 containerd:
  Version:           v2.3.4
  GitCommit:        db8809540e1a7a9da5d518876894933ff55692ab
 runc:
  Version:           1.5.1
  GitCommit:        v1.5.1-0-g8f2685a4
 docker-init:
  Version:           0.19.0
  GitCommit:        de40ad0
adam@Devsecops:~/docker-lab/bab-3$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
```

3.2 User-Defined Bridge Network dan DNS Service Discovery

Membuat bridge network lab-net (subnet 172.20.0.0/16) dan menjalankan dua container Nginx (server-a dan server-b). Perintah docker exec server-a ping -c 3 server-b membuktikan DNS service discovery berhasil menerjemahkan nama container secara dinamis

```
adam@Devsecops:~/docker-lab/bab-3$ docker network ls
NETWORK ID      NAME      DRIVER      SCOPE
041949e2b787    bridge    bridge      local
65ce1b444d2c    host      host        local
b0b29c665d76    lab-net   bridge      local
4784c62e83cc    none     null        local
adam@Devsecops:~/docker-lab/bab-3$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS      NAMES
4c756c257e4a   nginx:alpine  "/docker-entrypoint..." About a minute ago Up About a minute 80/tcp     server-b
77beeda4fee8   nginx:alpine  "/docker-entrypoint..." About a minute ago Up About a minute 80/tcp     server-a
adam@Devsecops:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-b
PING server-b (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.217 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.105 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.143 ms

--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.105/0.155/0.217 ms
```

3.3 Manajemen Data Container: Named Volume, Bind Mount, dan tmpfs

1. Inisiasi Named Volume 'data-vol'

Membuat volume data-vol dan menjalankan container writer untuk menguji penulisan log waktu secara kontinu ke dalam volume.

```
adam@Devsecops:~/docker-lab/bab-3$ docker volume ls
DRIVER      VOLUME NAME
local       data-vol
adam@Devsecops:~/docker-lab/bab-3$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Sat Sep 12 02:24:38 UTC 2026
Sat Sep 12 02:24:43 UTC 2026
Sat Sep 12 02:24:48 UTC 2026
Sat Sep 12 02:24:53 UTC 2026
Sat Sep 12 02:24:58 UTC 2026
Sat Sep 12 02:25:03 UTC 2026
Sat Sep 12 02:25:08 UTC 2026
Sat Sep 12 02:25:14 UTC 2026
Sat Sep 12 02:25:19 UTC 2026
```

2. Pembuktian Persistensi Data Named Volume

Setelah container writer dihapus paksa (docker rm -f writer), container baru dijalankan me-mount volume yang sama. Pembacaan file log.txt membuktikan data tetap persisten melampaui lifecycle container

```
adam@Devsecops:~/docker-lab/bab-3$ docker rm -f writer
writer
adam@Devsecops:~/docker-lab/bab-3$ docker volume ls
DRIVER      VOLUME NAME
local       data-vol
adam@Devsecops:~/docker-lab/bab-3$ docker run --rm \
-v data-vol:/data \
alpine:3.20 \
cat /data/log.txt
Sat Sep 12 02:24:38 UTC 2026
Sat Sep 12 02:24:43 UTC 2026
Sat Sep 12 02:24:48 UTC 2026
Sat Sep 12 02:24:53 UTC 2026
Sat Sep 12 02:24:58 UTC 2026
Sat Sep 12 02:25:03 UTC 2026
Sat Sep 12 02:25:08 UTC 2026
Sat Sep 12 02:25:14 UTC 2026
Sat Sep 12 02:25:19 UTC 2026
Sat Sep 12 02:25:24 UTC 2026
Sat Sep 12 02:25:29 UTC 2026
Sat Sep 12 02:25:34 UTC 2026
Sat Sep 12 02:25:39 UTC 2026
Sat Sep 12 02:25:44 UTC 2026
Sat Sep 12 02:25:49 UTC 2026
Sat Sep 12 02:25:54 UTC 2026
Sat Sep 12 02:25:59 UTC 2026
Sat Sep 12 02:26:04 UTC 2026
Sat Sep 12 02:26:09 UTC 2026
Sat Sep 12 02:26:14 UTC 2026
Sat Sep 12 02:26:19 UTC 2026
```

3. Backup Named Volume Menggunakan Helper Container

Menjalankan container sementara berbasis Alpine untuk mengompres direktori volume ke dalam file arsip tar: data-vol-backup.tar.gz pada sistem host.

```
adam@Devsecops:~/docker-lab/bab-3$ ls -lh backup/
total 4.0K
-rw-r--r-- 1 root root 217 Sep 12 09:29 data-vol-backup.tar.gz
```

4. Pengujian Bind Mount dengan Sinkronisasi Host

Memetakan direktori host ./html ke dalam container Nginx. Perubahan file HTML di host langsung ter-update secara real-time pada respon server tanpa rebuild image

```
adam@Devsecops:~/docker-lab/bab-3$ curl http://localhost:8080
<h1>Bab 3 DevSecOps</h1>
<p>File berubah dari host.</p>
```

5. Pengujian Karakteristik tmpfs (RAM-Based Storage)

Menguji penyimpanan memori non-persisten menggunakan tmpfs. Data di direktori tmpfs terbukti terhapus secara otomatis begitu container di-restart.

```
adam@Devsecops:~/docker-lab/bab-3$ docker exec tmpfs-test cat /tmp/test.txt
data sementara
adam@Devsecops:~/docker-lab/bab-3$ docker rm -f tmpfs-test
tmpfs-test
adam@Devsecops:~/docker-lab/bab-3$ docker run --rm \
  --tmpfs /tmp \
  alpine:3.20 \
  ls -la /tmp
total 4
drwxrwxrwt  2 root    root          40 Sep 12 02:39 .
drwxr-xr-x  1 root    root        4096 Sep 12 02:39 ..
```

3.4 Implementasi Stack Multi-Container dengan Docker Compose

1. Build Image Multi-Container Compose

Mengeksekusi docker compose build untuk membangun image custom service backend Flask sesuai konfigurasi Dockerfile dan dependencies.

```
adam@Devsecops:~/docker-lab/bab-3$ docker compose up -d --build
[+] Building 18.0s (12/12) FINISHED
=> [internal] load local base definitions 0.0s
=> reading from stdin 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> transferring Dockerfile: 200B 0.0s
=> [internal] load metadata for docker.io/library/python:3.12-alpine 1.4s
=> [internal] load .dockerignore 0.0s
=> transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.12-alpine@sha256:b04631e04e492010c50fb6dddf028f7f35f0e7425cb44aa09bca53e708a35a 0.1s
=> resolve docker.io/library/python:3.12-alpine@sha256:b04631e04e492010c50fb6dddf028f7f35f0e7425cb44aa09bca53e708a35a 0.1s
=> [internal] load build context 0.0s
=> transferring context: 63B 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY requirements.txt 0.0s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt 15.4s
=> [5/5] COPY app.py 0.3s
=> exporting to image 5.0s
=> exporting layers 2.9s
=> exporting manifest sha256:ddebed7efed22b0af7059453506304265354aa3fd5248db8b633cbb41307 0.1s
=> exporting config sha256:d2a9efase71f3750820f390f7820caesf6b7d0b223c7e0dbb0f85df7058820f 0.1s
=> exporting attestation manifest sha256:6af138040377234299ee23183ff11cdcf0158b0c09ceaf2e3b9d2d3442 0.1s
=> exporting manifest list sha256:3eb3caf9cb91bc90ab055aff11a3e08efe9e90a5e2e9a0e911b71b237f4af 0.1s
=> naming to docker.io/library/bab-3-app:latest 0.0s
=> unpacking to docker.io/library/bab-3-app:latest 1.0s
=> resolving provenance for metadata file 0.1s
[+] up 7/7
✔ Image bab-3-app Built 18.4s
✔ Network bab-3_backend Created 0.4s
✔ Volume bab-3_pg-data Created 0.0s
✔ Network bab-3_frontend Created 0.2s
✔ Container bab-3-db-1 Healthy 13.0s
✔ Container bab-3-app-1 Started 12.0s
✔ Container bab-3-web-1 Started 11.2s
```

3.5 Verifikasi dan Pengujian Layanan

1. Pemeriksaan Status Stack dengan 'docker compose ps'
Memverifikasi ketiga service (web, app, db) berjalan normal. Service db berhasil mencapai status healthy melalui pengujian berkala pg_isready

```
adam@DevSecops:~/docker-lab/bab-3$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
bab-3-app-1	bab-3-app	"python app.py"	app	50 seconds ago	Up 37 seconds	5000/tcp
bab-3-db-1	postgres:16-alpine	"docker-entrypoint.s..."	db	50 seconds ago	Up 49 seconds (healthy)	5432/tcp
bab-3-web-1	nginx:alpine	"/docker-entrypoint..."	web	49 seconds ago	Up 37 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/t

2. Pengujian Web Ingress Nginx via cURL
Mengeksekusi curl -v http://localhost:8080 pada host. Respon 200 OK dengan konten HTML dari bind mount membuktikan reverse proxy bekerja sempurna.

```
adam@DevSecops:~/docker-lab/bab-3$ curl -v http://localhost:8080
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET / HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: nginx/1.31.5
< Date: Sat, 12 Sep 2026 03:11:45 GMT
< Content-Type: text/html
< Content-Length: 229
< Last-Modified: Sat, 12 Sep 2026 02:50:30 GMT
< Connection: keep-alive
< ETag: "6aa4bdf6-e5"
< Accept-Ranges: bytes
<
<!DOCTYPE html>
<html>
<head>
  <title>DevSecOps Bab 3</title>
</head>
<body>
  <h1>Docker Compose Berhasil</h1>
  <p>Nginx Web Container berjalan.</p>
</body>
</html><h1>Bab 3 DevSecOps</h1>
<p>File berubah dari host.</p>
* Connection #0 to host localhost left intact
```

3. Verifikasi Endpoint API Flask dan Integrasi Database
Menguji endpoint internal via wget: Endpoint / merespons status: ok dan /health membuktikan database PostgreSQL berhasil terhubung.

```
adam@DevSecops:~/docker-lab/bab-3$ docker compose exec app wget -qO- http://127.0.0.1:5000/
{"message":"Flask App berjalan","status":"ok"}
adam@DevSecops:~/docker-lab/bab-3$ docker compose exec app wget -qO- http://127.0.0.1:5000/health
{"database":"connected","status":"healthy"}
```

3.6 Analisis Wajib dan Troubleshooting

1. Error Build Compose: File requirements.txt Not Found
Kegagalan proses build backend pada Dockerfile step COPY requirements.txt . akibat file requirements.txt belum tersedia pada build context

```
adam@DevSecops:~/docker-lab/bab-3$ docker compose build --no-cache
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
bab-3-app-1	bab-3-app	"python app.py"	app	50 seconds ago	Up 37 seconds	5000/tcp
bab-3-db-1	postgres:16-alpine	"docker-entrypoint.s..."	db	50 seconds ago	Up 49 seconds (healthy)	5432/tcp
bab-3-web-1	nginx:alpine	"/docker-entrypoint..."	web	49 seconds ago	Up 37 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/t

```
-----
[+] up 13/14v requirements.txt :
  Image postgres:16-alpine failed
  Image bab-3-app building
dockerfile:5
-----
3 | WORKDIR /app
4 |
5 | >>> COPY requirements.txt .
6 |
7 | RUN pip install --no-cache-dir -r requirements.txt
-----
failed to solve: failed to compute cache key: failed to calculate checksum of ref vug6s4lo86iaq5gsl000bbjq:pjgtitgtogk30p4myo2an9ieb: "/requirements.txt": not found
```

Masalah yang Muncul dan Metode Diagnosis:

- Gejala: Eksekusi docker compose up mengalami kegagalan pada build context service app dengan pesan failed cache key: failed to calculate checksum: '/requirements.txt': not found.
- Diagnosis: Pemeriksaan Dockerfile pada step 5 menemukan instruksi COPY requirements.txt ., namun direktori ./app pada host belum memiliki file tersebut.
- Solusi: Membuat file requirements.txt berisi paket flask dan psycopg2-binary pada direktori ./app, kemudian mengulang build dengan docker compose build --no-cache.

BAB IV EVALUASI DAN LATIHAN MANDIRI

1. Mengapa user-defined bridge lebih baik daripada default bridge untuk multi-container app?

User-defined bridge menyediakan resolusi DNS otomatis berdasarkan nama kontainer/service, menyediakan isolasi jaringan tersegregasi secara default tanpa koneksi liar antar kontainer independen, serta memungkinkan penyesuaian parameter subnet dan gateway secara fleksibel.

2. Apa risiko bind mount terhadap keamanan host?

Bind mount menghubungkan sistem berkas container langsung ke direktori host. Jika container disusupi atau berjalan sebagai root tanpa opsi read-only (:ro), penyerang dapat memodifikasi, menimpa file konfigurasi sistem operasi host, atau menyisipkan skrip berbahaya.

3. Apa perbedaan docker compose down dan docker compose down -v?

docker compose down hanya menghentikan dan menghapus kontainer serta jaringan internal stack, sementara named volume data tetap utuh. Sebaliknya, docker compose down -v secara destruktif turut menghapus named volume beserta seluruh data persisten di dalamnya.

4. Kapan depends_on dengan healthcheck lebih tepat daripada depends_on biasa?

Digunakan ketika service dependen (seperti backend Flask) membutuhkan service prasyarat (seperti PostgreSQL) dalam kondisi siap menerima koneksi (ready state). depends_on standar hanya menunggu proses container dijalankan, bukan kesiapan layanannya.

5. Bagaimana strategi backup volume untuk database produksi?

Strategi backup produksi meliputi: (a) Melakukan logical dump transaksional menggunakan pg_dump secara berkala; (b) Snapshot storage volume secara konsisten; (c) Enkripsi dan replikasi file backup ke storage sekunder/cloud terpisah; (d) Menjalankan uji pemulihan (restore drill) secara rutin

BAB V KESIMPULAN

Praktikum Bab 3 telah berhasil mengonseptualisasikan dan mengimplementasikan pilar fundamental jaringan, persistensi data, dan orkestrasi multi-container pada Docker. Penggunaan user-defined bridge membuktikan name resolution otomatis mampu menghadirkan service discovery dinamis yang tangguh. Pemisahan mekanisme storage (Volume, Bind Mount, dan tmpfs) berhasil diterapkan sesuai karakteristik beban kerja data. Implementasi Docker Compose dengan pembagian network tier (frontend dan backend) serta integrasi healthcheck memastikan ketergantungan antar komponen tersinkronisasi dengan stabil dan aman. Penerapan analisis risiko keamanan dan rekomendasi arsitektural menjadi bekal penting dalam merancang pipeline DevSecOps yang aman, andal, dan siap produksi